LAPORAN AKHIR
IMPLEMENTASI *REAL-TIME MULTIBAND EQUALIZER* BERBASIS MATLAB

Kelompok 9

Muhammad Nauval Ferry Widya Akmal	**2306151161**
Steven Wijaya Lim	**2306215122**
Muhammad Dimas Fachrizki	**2306246065**

UNIVERSITAS INDONESIA

DEPOK

2025

ABSTRAK

Pengolahan sinyal digital memegang peranan vital dalam sistem audio modern, khususnya dalam pengembangan *equalizer* untuk manipulasi spektrum frekuensi. Proposal ini merancang dan mengimplementasikan sistem *Real-Time Multiband Equalizer* berbasis perangkat lunak MATLAB. Tujuan utama penelitian ini adalah menciptakan sistem pemrosesan audio yang responsif dengan latensi rendah menggunakan arsitektur hibrida serial. Sistem menggabungkan dua jenis filter digital: filter FIR (*Finite Impulse Response*) orde 512 sebagai *Graphic Equalizer* utama dengan metode desain paralel untuk berbagai *preset* (Rock, Jazz, Pop, dan Flat), serta filter IIR (*Infinite Impulse Response*) Butterworth orde 4 sebagai *Low-Pass Filter* dengan frekuensi *cut off* 8 kHz untuk pembatasan *bandwidth* dan reduksi *noise*. Implementasi dilakukan menggunakan pemrosesan berbasis *frame* (1024 sampel) dan antarmuka *Command Line Interface* (CLI) untuk meminimalkan beban komputasi. Hasil analisis menunjukkan bahwa sistem mampu beroperasi secara *real-time* dengan latensi filter teoretis sekitar 6,5 ms, menjadikannya solusi yang efektif dan efisien untuk aplikasi *playback* audio digital.

Kata Kunci: *Equalizer, FIR, IIR, MATLAB, Real-Time Signal Processing.*

DAFTAR ISI

ABSTRAK **1**
DAFTAR ISI **2**
BAB I PENDAHULUAN **4**
A. Latar Belakang Masalah 4
B. Perumusan Masalah 5
C. Tujuan dan Manfaat 5
BAB II KAJIAN PUSTAKA **6**
A. Filter Digital dan Equalizer Parametrik 6
B. *Sampling* dan *Frames* 6
C. Z-*transformation* 7
D. FIR Filter 7
E. IIR Filter 8
BAB III METODE PENULISAN **9**
A. Arsitektur Sistem 9
B. Perancangan Filter *Hybrid* 9
C. Implementasi *Software* 10
DAFTAR PUSTAKA **11**

BAB I
PENDAHULUAN

A. Latar Belakang Masalah

Perkembangan teknologi pengolahan sinyal digital telah memberikan dampak yang besar dalam berbagai aplikasi. Mulai dari *equalizer* musik, produksi audio, live sound, sampai sistem komunikasi suara. Pada sistem audio, salah satu teknik pemrosesan penting adalah *multiband filtering*. Proses ini memisahkan audio ke beberapa rentang frekuensi lalu tiap *band* bisa diolah secara independen tanpa mengubah frekuensi yang lainnya. Teknik ini juga sering dipakai di perangkat profesional seperti *parametric equalizer, multiband compressor* sampai *noise reduction system*, karena bisa membuat suara lebih bersih dan juga mengurangi gangguan di frekuensi tertentu.

Parametric multiband memberikan fleksibilitas dan kontrol penuh terhadap pengguna. Sistem ini memiliki keunggulan fleksibilitas dan penyesuaian karakteristik frekuensi melalui parameter seperti gain, frekuensi pusat, dan *bandwidth (Q-factor)*. Dengan fleksibilitas ini, pengguna dapat mengatur spektrum audio sampai detail, persis seperti karakter suara yang diinginkan.

Banyak aplikasi yang membutuhkan proses respons langsung input suara pengguna, kemampuan sistem untuk bekerja secara *real time.* Sistem tidak hanya harus memberikan produk yang berkualitas, namun juga harus memiliki *latency/delay* yang serendah mungkin agar perubahan tiap parameter dapat didengar seketika. Maka dari itu, pemilihan platform dan teknik dari desain sistem *digital signal processing* menjadi aspek yang penting dalam keberhasilan implementasi.

MATLAB menjadi platform yang banyak digunakan dalam kebutuhan ini. Fitur-fitur di MATLAB seperti desain filter, visualisasi sinyal, dan integrasi audio *real-time* dapat dimanfaatkan menjadi sistem *multiband real-time* yang efisien baik dari sisi desain algoritma maupun *interface* pengguna. Melalui *project* ini, kita tidak hanya mempelajari konsep dari *filter* digital, pembagian frekuensi dan *equalizer* parametrik. Tapi kita juga langsung menerapkan pemrosesan *real-time* dan merancang *interface* yang jelas sesuai kebutuhan pengguna.

B. Perumusan Masalah

1. Bagaimana merancang sistem pemrosesan sinyal *multiband* berbasis MATLAB yang mampu membagi sinyal audio ke beberapa frekuensi *band?*
2. Bagaimana menerapkan kontrol parameter seperti *gain* dan *bandwidth/Q-factor* pada setiap *band* secara efektif?
3. Bagaimana memastikan pemrosesan sinyal berjalan secara real time dengan *latency/delay* yang rendah dan kualitas audio yang baik?
4. Bagaimana melakukan evaluasi performa sistem, baik dari aspek respons frekuensi, stabilitas, maupun kualitas audio hasil pemrosesan?

C. Tujuan dan Manfaat

1. Tujuan utama *project* ini adalah mengembangkan sistem pemrosesan *audio multiband* yang mampu bekerja secara realtime menggunakan MATLAB.
2. Menerapkan *filter digital* serta pengaturan parameter frekuensi, *gain,* dan *Q-factor* untuk menghasilkan kontrol spektrum audio yang presisi dan fleksibel.
3. Mengoptimalkan performa pemrosesan agar perubahan parameter dapat diterapkan tanpa *latency/delay* yang mengganggu dan tetap menjadi kualitas *audio* yang baik.

BAB II
LANDASAN TEORI

A. Filter Digital dan Equalizer Parametrik

Equalization adalah proses fundamental dalam pengolahan audio yang bertujuan untuk mengubah respons frekuensi sinyal, baik untuk memperkuat (boost) atau melemahkan (attenuate) pita frekuensi tertentu. Walaupun terdapat berbagai jenis *equalizer* (seperti grafis atau *shelving*), equalizer parametrik memberikan kontrol yang paling detail. Jenis ini memungkinkan pengguna untuk mengatur secara presisi tiga parameter utama pada pita frekuensi yang dipilih frekuensi tengah (*centre frequency*), *bandwidth* (lebar pita), dan *gain* (penguatan atau atenuasi) [1].

Implementasi *equalizer* modern banyak dilakukan secara digital karena keunggulan yang ditawarkan oleh filter digital dibandingkan sirkuit analog. Pendekatan digital memungkinkan perancangan filter dengan orde yang sangat tinggi serta desain respons frekuensi yang arbitrer (spesifik). Kemampuan ini sangat penting untuk ekualisasi presisi dan sulit dicapai menggunakan komponen analog, sehingga menjadikan *filtering digital* sebagai landasan teori utama dalam implementasi equalizer parametrik [1].

B. *Sampling* dan *Frames*

1. *Sampling*:

   Proses di mana sinyal analog diubah menjadi digital disebut *sampling*. Mencuplik sinyal kontinu berarti mengukur sinyal tersebut secara periodik, kemudian membulatkan dan menyimpan nilai-nilai yang terukur. Saat memproses sinyal audio, frekuensi sampling yang memadai harus dipilih untuk menghindari aliasing. Teorema *sampling* Nyquist-Shannon menyatakan bahwa laju sampling minimal yang diperlukan untuk merekonstruksi sinyal asli secara sempurna adalah dua kali frekuensi komponen maksimum dari sinyal yang disampling [2].

   Teorema Nyquist-Shannon menyatakan bahwa frekuensi sampling yang digunakan harus setidaknya dua kali lebih besar dari frekuensi komponen maksimum yang ada dalam sinyal asli [2].

$$f_s \geq 2f_{max} \tag{1}$$

Batas atas spektrum pendengaran manusia secara umum dianggap sekitar 20 kHz. Oleh karena itu, untuk merekonstruksi sinyal audio secara sempurna, frekuensi sampling standar industri (seperti pada CD audio) ditetapkan pada 44.1 kHz (44100 Hz) [2].

2. *Frame*

Dalam sistem *real-time*, memproses sinyal sampel demi sampel bisa jadi tidak efisien karena *overhead* komputasi yang tinggi (misalnya, memanggil fungsi filter untuk setiap sampel). Untuk mengatasi ini, sistem audio modern menggunakan pemrosesan berbasis *frame* [2].

Sebuah *frame* adalah sebuah *array* atau blok yang berisi sejumlah sampel audio yang dikumpulkan. Sistem akan menunggu hingga *frame* ini penuh, kemudian memproses seluruh *frame* tersebut sekaligus dengan FIR atau IIR [2].

C. Z-Transform

Pemahaman mengenai Z-*Transform* sangat penting, karena transformasi ini digunakan secara luas dalam berbagai teori *digital filtering*. Transformasi-z serupa dengan Transformasi Laplace, namun dikhususkan untuk sinyal waktu diskrit [3].

$$Z\{x[n]\} = X(z) = \sum_{n=0}^{\infty} x[n]z^{-n} \quad (2)$$

$$z = re^{j\omega} \quad (3)$$

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j\omega n} \quad (4)$$

Dapat dilihat bahwa jika variabel z pada persamaan (1) diganti dengan $e^{j\omega}$, persamaan tersebut akan berubah menjadi persamaan *Discrete Fourier Transform* pada persamaan (4). Ini setara dengan membatasi nilai z pada persamaan (2) dengan radius r = 1. Oleh karena itu, ketika melakukan analisis Transformasi-z menggunakan bidang-z, lingkaran satuan (*unit circle*) menjadi salah satu aspek analisis yang paling utama [3].

D. FIR Filter

Istilah FIR (*Finite Impulse Response*) mengacu pada filter digital yang fungsi transfernya bergantung pada kombinasi linier dari sinyal-sinyal *input* sebelumnya. Fungsi transfer H(z) untuk filter FIR umum diberikan sebagai berikut [2,3].

$$H(z) = \sum_{n=0}^{N-1} b_n z^{-n} \quad (5)$$

Di mana b_n adalah koefisien untuk setiap n dan $N < \infty$. Persamaan *output* dari filter FIR dalam domain waktu adalah akibat turunan dari persamaan (5), yang ditulis sebagai berikut [2,3].

$$y[k] = \sum_{n=0}^{N-1} b_n x[k - n] \quad (7)$$

Filter FIR dikarakterisasi oleh stabilitas, fasa linear, dan desain yang mudah, seringkali membutuhkan orde filter yang tinggi dan komputasi dalam jumlah besar yang dapat menjadi masalah jika diperlukan aplikasi *real-time*. Filter IIR membutuhkan koefisien filter yang lebih sedikit dan beban komputasi yang lebih ringan tetapi dapat menghasilkan distorsi fasa. Selain itu, filter FIR cukup sensitif terhadap kuantisasi koefisien dan efek dari panjang kata (*word length*) harus dipertimbangkan [3].

E. IIR Filter

Berbeda dengan filter FIR, fungsi transfer dari filter IIR (*infinite impulse response*) bergantung pada sinyal *input* dan *output* sebelumnya. Fungsi transfer dari filter IIR umum diberikan sebagai berikut [2,3].

$$H(z) = \frac{\sum_{n=0}^{N} b_n z^{-n}}{1 + \sum_{m=1}^{M} a_m z^{-m}} \quad (5)$$

di mana a_m dan b_m adalah koefisien unik untuk setiap m dan n. Persamaan keluaran (*output*) yang sesuai dapat dihasilkan melalui transformasi dari persamaan (5) ke domain waktu dengan menerapkan Transformasi-z balikan (*inverse Z-transform*), yang menghasilkan *output* berikut. [2,3].

$$y[k] = \sum_{n=0}^{N} b_n x[k - n] - \sum_{m=1}^{M} a_m y[k - m] \quad (7)$$

Strategi paling umum untuk filter IIR adalah; transformasi z-bilinear (yang dimulai dengan desain filter analog), teknik penempatan *pole-zero*, transformasi frekuensi, dan penyaringan *biquad*. Teknik desain filter FIR: metode *window*, metode *sampling* frekuensi, dan *inverse filtering*. [3].

BAB III
PERANCANGAN DAN IMPLEMENTASI SISTEM

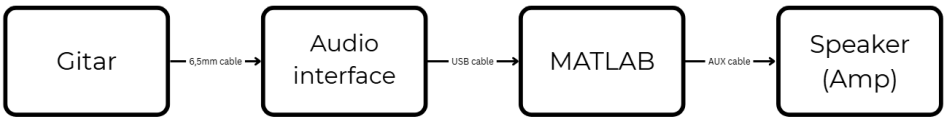
A. Alat dan Bahan

Dalam pengembangan sistem Real-Time Multiband Equalizer ini digunakan sejumlah hardware untuk melakukan proses akuisisi, pemrosesan, dan output sinyal audio digital. Spesifikasi *hardware device* yang digunakan adalah sebagai berikut:

1. Laptop dengan MATLAB terinstal
2. Pedal Gitar
3. Gitar Listrik
4. Amplifier Gitar (15 Watt)
5. Kabel Instrumen (Jack 6.5 mm)
6. Kabel Micro USB
7. Kabel AUX (Jack 3.5 mm)
8. *Headphone*

B. Konfigurasi Sistem

Untuk memungkinkan pemrosesan sinyal secara *real-time*, seluruh perangkat keras dihubungkan secara serial membentuk satu rantai sinyal utuh. Diagram alur koneksi fisik ditunjukkan pada Gambar 3.1.



Gambar 3.1 Diagram alur koneksi fisik *Real-Time Multiband Equalizer* Berbasis MATLAB

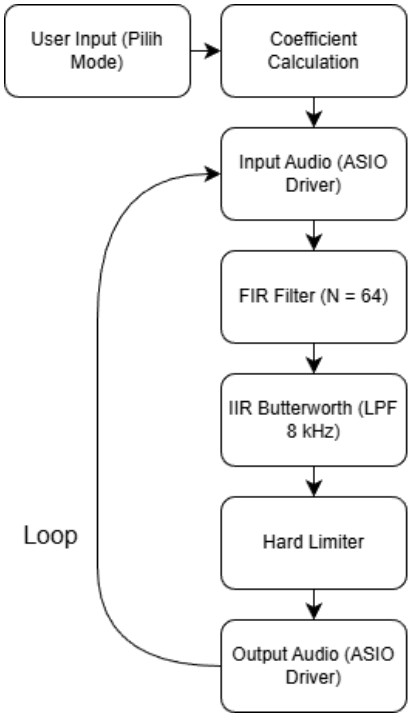
Alur sinyal fisik berjalan sebagai berikut:

1. Sinyal analog dari gitar dikirim melalui kabel instrumen 6.5 mm menuju *input* pada *audio interface* pedal gitar.
2. *Audio interface* mengonversi sinyal analog menjadi data digital dan mengirimkannya ke laptop melalui koneksi kabel USB.
3. Setelah diproses oleh MATLAB di laptop, sinyal digital dikonversi kembali menjadi analog oleh kartu suara laptop dan dikirimkan melalui *output headphone* (kabel AUX 3.5 mm).
4. Sinyal akhir diperkuat dan dibunyikan oleh *speaker* (*amplifier*).

Sistem *software* dirancang menggunakan platform MATLAB dengan memanfaatkan *Audio System Toolbox* dan DSP *System Toolbox*. Arsitektur sistem difokuskan pada pemrosesan sinyal waktu nyata (*real-time*) dengan prioritas pada latensi rendah (*low latency*) dan kejernihan sinyal.

Untuk mengatasi masalah latensi tinggi yang umum terjadi pada *driver* audio standar sistem operasi (DirectSound/MME), sistem ini mengimplementasikan protokol ASIO (Audio Stream Input/Output). Dengan *driver* tambahan ASIO4ALL v2 memungkinkan akses langsung antara *software* MATLAB dan *hardware* audio, melewati lapisan *mixer* sistem operasi.

Ukuran *buffer* ditetapkan pada nilai rendah (128-512 sampel) untuk meminimalkan jeda waktu antara input dan output. Pemilihan ukuran *buffer* ini didasarkan pada keseimbangan antara responsivitas sistem dan stabilitas aliran data agar tidak terjadi *dropout* (suara terputus).



Gambar 3.2 *Flowchart* proses program kerja *Real-Time Multiband Equalizer* Berbasis MATLAB

C. Perancangan Filter *Hybrid*

Inti dari pemrosesan sinyal digital pada sistem menerapkan arsitektur hibrida yang menggabungkan dua jenis filter secara serial untuk mendapatkan karakteristik tonal yang sesuai namun tetap efisien dari segi komputasi.

1. Tahap 1

Tahap pertama pemrosesan sinyal difokuskan pada pembentukan respon frekuensi utama menggunakan filter jenis Finite Impulse Response (FIR) dengan orde filter N = 64. Pada rancangan awal, sistem direncanakan menggunakan orde filter yang lebih tinggi (N = 512), namun dalam implementasi akhir, orde filter direduksi menjadi 64. Penyesuaian ini dilakukan untuk mengoptimalkan efisiensi komputasi dan meminimalkan latensi sistem agar dapat beroperasi secara *real-time* dengan responsivitas tinggi, tanpa mengorbankan kualitas pembentukan nada.

Metode desain yang diterapkan menggunakan pendekatan paralel, di mana respons frekuensi total dibentuk dengan menjumlahkan respons dari beberapa filter *band-pass* yang bekerja secara bersamaan. Sistem membagi spektrum audio menjadi lima titik frekuensi pusat, yaitu 1000 Hz, 3000 Hz, 5000 Hz, 8000 Hz, dan 16000 Hz. Koefisien filter dihitung secara dinamis berdasarkan nilai penguatan (*gain*) yang telah ditentukan untuk setiap preset genre (Rock, Jazz, Pop, dan Flat).

Dalam implementasi teknisnya, algoritma menggunakan variabel akumulator (b_total) yang menggabungkan karakteristik dari seluruh pita frekuensi menjadi satu set koefisien filter FIR tunggal. Pendekatan ini memungkinkan sistem untuk hanya melakukan satu kali proses konvolusi pada saat *run-time*, sehingga beban pemrosesan sinyal menjadi jauh lebih ringan dibandingkan memproses setiap pita frekuensi secara terpisah.

2. Tahap 2

Tahap kedua dilanjutkan dengan penerapan filter *Infinite Impulse Response* (IIR) jenis Butterworth orde 4 dengan frekuensi *cut-off* 8 kHz. Filter ini berfungsi sebagai *clean-up filter* yang membatasi *bandwidth* sinyal keluaran agar energi suara tetap terfokus pada rentang frekuensi audio utama sekaligus meredam *noise* digital bernada tinggi (*hiss*). Dalam implementasinya, fungsi butter pada MATLAB menghasilkan vektor koefisien pembilang (b_iir) dan penyebut (a_iir) melalui proses normalisasi frekuensi *cut-off* terhadap frekuensi Nyquist (fs/2). Sinyal keluaran dari tahap ini kemudian dibatasi amplitudonya untuk mencegah terjadinya *clipping* sebelum dikirim ke *interface output* audio.

```
Orde_iir = 4;
Fc_iir = 8000;
Wn_iir = Fc_iir / (fs/2);
[b_iir, a_iir] = butter(Orde_iir, Wn_iir, 'low');
```

BAB IV
HASIL DAN PEMBAHASAN

A. Hasil dan Pembahasan

Sistem *Real-Time Equalizer* berbasis MATLAB bertujuan untuk memindahkan pemrosesan sinyal efek digital yang biasanya dilakukan oleh *hardware* (*pedal stompbox*) atau DAW (*Digital Audio Workstation*) ke dalam lingkungan komputasi brbasis MATLAB. Sistem ini mengintegrasikan instrumen fisik (gitar elektrik) dengan *software* melalui perantara *audio interface* eksternal. Dalam skenario pengujian, *audio interface* berfungsi sebagai perantara yang mengubah sinyal analog menjadi data digital agar dapat diolah oleh komputer, dan kemudian mengembalikannya menjadi sinyal analog menuju *amplifier*.

Berdasarkan hasil uji coba, sistem mampu beroperasi sesuai logika perancangan. Tahap awal pengujian difokuskan pada sinkronisasi perangkat. Menggunakan kode untuk mendeteksi *audio interface,* MATLAB berhasil mengenali driver sebagai perangkat input utama. Hasil ini cukup penting untuk memastikan bahwa sinyal yang diproses benar-benar berasal dari gitar, bukan dari mikrofon bawaan laptop.

Efektivitas algoritma filter diuji dengan mengubah-ubah mode ekualisasi yang tersedia, yaitu *Rock, Jazz, Pop, dan Flat.* Secara visual dan auditori, perubahan mode ini memberikan dampak:

1. Mode *Rock*: Grafik menunjukkan karakteristik *"V-shape"* atau *scooped*, di mana frekuensi menengah (1–3 kHz) diredam, sementara frekuensi rendah (Bass) dan tinggi (Treble) diperkuat. Hal ini sesuai dengan kebutuhan genre *Rock* untuk menghasilkan suara distorsi yang bertenaga.
2. Mode *Jazz*: Terlihat adanya penurunan (*roll-off*) yang halus pada frekuensi tinggi. Karakteristik ini menciptakan nuansa suara yang "bulat" (*warm*) dan menghilangkan ketajaman (*harshness*) yang tidak diinginkan, sesuai dengan karakteristik gitar *Jazz* tradisional.
3. Mode *Pop*: Kurva respons difokuskan pada pengangkatan artikulasi nada menengah (*mid-range*). Filter ini berhasil membuat suara gitar terdengar lebih *crisp* (renyah) dan menonjol di antara instrumen lain.
4. Efek Filter IIR (*Low-Pass*): Pada seluruh mode pengujian, terlihat adanya penurunan drastis magnitudo pada frekuensi di atas 8 kHz. Ini membuktikan bahwa tahap kedua (*Stage 2*) filter IIR Butterworth bekerja efektif

memangkas *noise* digital dan frekuensi tinggi yang tidak diperlukan, menjaga sinyal output tetap bersih.

Selain magnitudo, pengujian juga memantau respons fasa. Grafik *Phase Response* menunjukkan pola linier (*linear phase*) pada rentang frekuensi kerja audio. Hal ini mengindikasikan bahwa penggunaan filter FIR pada tahap pembentukan nada berhasil menjaga integritas sinyal transien (seperti serangan petikan senar), sehingga sinyal tidak mengalami cacat waktu (*time-smearing*) yang sering terjadi pada equalizer analog konvensional.

B. Pembahasan Algoritma Program

```
reader = audioDeviceReader('Driver', 'ASIO', ...
    'Device', 'ASIO4ALL v2', ...
    'SamplesPerFrame', 128, ...
    'SampleRate', 44100);
```

Segmen kode di atas adalah kunci utama performa sistem. Berbeda dengan pendekatan standar yang menggunakan *driver* Windows (DirectSound), sistem ini memanggil driver ASIO. Driver 'ASIO' memungkinkan MATLAB melakukan *bypass* terhadap *mixer* sistem operasi, sehingga sinyal audio dapat masuk dan keluar dengan jalur terpendek. SamplesPerFrame (128) sebagai ukuran *buffer* ditetapkan pada angka rendah (128 sampel). Secara teoritis, pada *sampling rate* 44100 Hz, *buffer* ini menghasilkan latensi dasar sekitar 2,9 ms. Nilai ini dipilih sebagai titik optimal di mana latensi tidak terdengar oleh telinga manusia (*imperceptible*), namun aliran data tetap stabil tanpa *dropout* (suara kresek).

```
% Stage 1: FIR Tone Shaping
firFilter = dsp.FIRFilter('Numerator', b_fir);
% Stage 2: IIR Noise Cleaning
iirFilter = dsp.IIRFilter('Numerator', b_iir, 'Denominator',
a_iir);
```

Berbeda dengan fungsi filter biasa, sistem ini menggunakan DSP System Objects (dsp.FIRFilter dan dsp.IIRFilter). Program internal MATLAB ini dirancang khusus untuk pemrosesan *streaming*. Algoritma ini tidak menghitung ulang koefisien setiap saat, melainkan menyimpan "state" (kondisi) filter di memori, sehingga proses eksekusi jauh lebih ringan dan efisien untuk aplikasi *real-time*.

```
while true
      audioIn = reader();              % 1. Akuisisi Data
      audioMid = firFilter(audioIn);   % 2. Stage 1 (Tone)
      audioOut = iirFilter(audioMid); % 3. Stage 2 (Cleanup)
```

```
    writer(audioOut);                % 4. Output Audio
end
```

Blok kode *infinite loop* (while true) berfungsi sebagai program utama sistem yang menjalankan siklus pemrosesan sinyal secara kontinu. Di dalam siklus ini, terjadi alur pemrosesan serial yang dimulai dengan akuisisi data audio mentah dari *buffer* driver ASIO. Proses selanjutnya menerapkan metode pemfilteran bertingkat, di mana sinyal diolah secara berurutan melewati dua tahap krusial pertama, objek firFilter bekerja membentuk karakteristik warna suara (*tone shaping*), yang kemudian langsung diteruskan ke objek iirFilter untuk membersihkan residu sinyal atau *noise*. Siklus diakhiri dengan pengiriman sinyal output yang telah terproses kembali ke perangkat keras audio. Seluruh rangkaian operasi ini dieksekusi dalam orde milidetik untuk setiap *frame*, menciptakan persepsi pemrosesan instan tanpa jeda (*real-time*) bagi pengguna.

```
function b_total = ekualisasiParalel(F, G_dB, N, Fm)
    G_linear = db2mag(G_dB);
    F_norm = F / (Fm/2);
    b_total = fir1(N, F_norm(1), 'low') * G_linear(1);
    for i = 1:length(F)-1
        b_bp = fir1(N, [F_norm(i), F_norm(i+1)], 'bandpass') *
G_linear(i+1);
        b_total = b_total + b_bp;
    end
    b_hp = fir1(N, F_norm(end), 'high') * G_linear(end);
    b_total = b_total + b_hp;
end
```

Kode fungsi ekualisasiParalel ini bertujuan menyusun satu filter FIR gabungan (b_fir) yang merepresentasikan karakteristik suara *genre* tertentu (seperti Rock, Jazz, atau Pop) dengan memanfaatkan prinsip superposisi, di mana rentang frekuensi dipisah-pisah dan dikuatkan secara independen sebelum disatukan kembali. Prosesnya dimulai dengan mengonversi nilai *gain* pada array konstanta *genre* dari desibel ke skala linear, lalu fungsi secara bertahap membuat filter *low-pass* untuk frekuensi bawah (0–1000 Hz), melakukan *looping* untuk membuat serangkaian filter *band-pass* pada celah frekuensi tengah (1000–3000 Hz, 3000–5000 Hz, dst.), serta filter *high-pass* untuk frekuensi atas (>16000 Hz), yang mana koefisien dari setiap filter tersebut langsung dikalikan dengan nilai *gain* linear yang bersesuaian pada indeks array *genre* tersebut. Akhirnya, seluruh koefisien filter terpisah yang telah diamplifikasi volumenya ini dimasukkan sebagai array ke dalam variabel b_total, menghasilkan sebuah array koefisien filter tunggal yang secara efisien membentuk

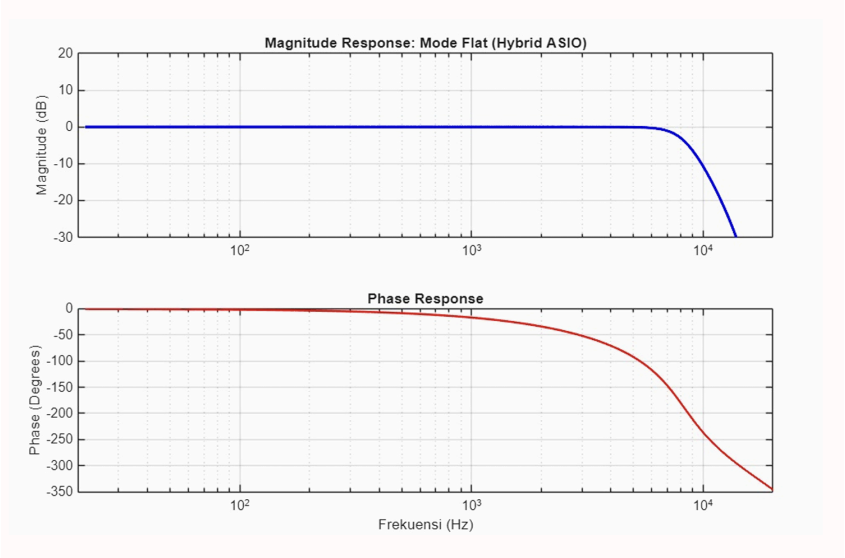
respons frekuensi audio sesuai profil *genre* yang dipilih dalam sekali proses penyaringan.

```
if strcmpi(Mode_Nama, 'Flat')
    b_fir = 1;
else
      b_fir = ekualisasiParalel(frekuensi, Mode_Aktif, N_FIR,
fs);
end
[b_iir, a_iir] = butter(N_IIR, Fc_IIR/(fs/2), 'low');
```

Segmen kode program ini berperan sebagai pengambil keputusan utama untuk konfigurasi filter digital, dimulai dengan memeriksa apakah mode suara yang aktif adalah Flat menggunakan fungsi perbandingan teks, yang jika benar maka koefisien filter FIR akan di *set* menjadi nilai tunggal satu agar sinyal audio diteruskan murni tanpa perubahan ekualisasi, namun jika mode lain yang dipilih maka fungsi ekualisasi paralel akan dijalankan untuk menyusun koefisien filter sesuai profil genre musik tersebut. Setelah tahapan penentuan filter FIR selesai, program secara otomatis menghitung koefisien filter IIR menggunakan desain Butterworth untuk membentuk filter lolos bawah atau *low pass* filter yang bertujuan memangkas frekuensi yang terlalu tinggi demi menjaga kualitas sinyal akhir tetap bersih dan stabil. Fungsi butter dipanggil untuk menghitung dan menghasilkan dua set koefisien penting yaitu b_iir sebagai pembilang dan a_iir sebagai penyebut yang merepresentasikan mekanisme umpan balik atau feedback khas filter IIR. Nilai koefisien-koefisien ini diturunkan secara matematis dari variabel N_IIR yang menentukan orde atau tingkat kompleksitas filter yang mempengaruhi seberapa tajam kemiringan pemotongan frekuensi yang dihasilkan.

C. Analisis Grafik

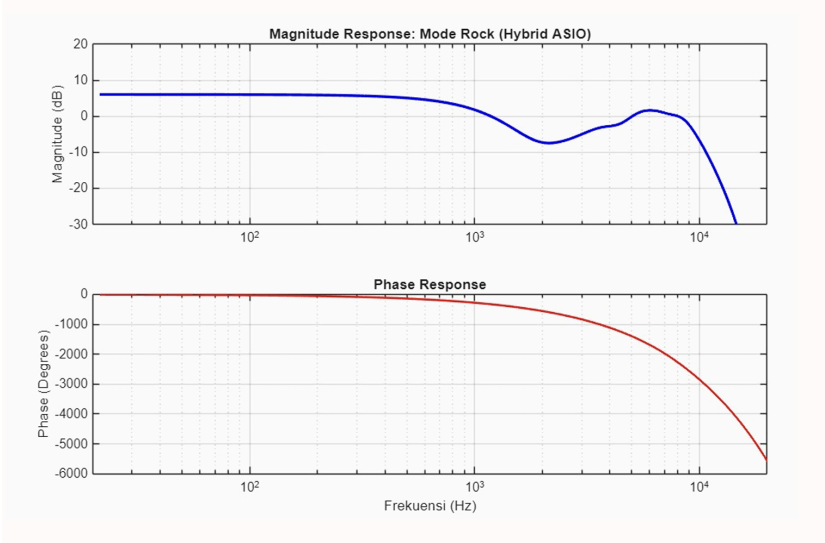
1. Mode *Flat*



Gambar 4.1 Grafik Respon Frekuensi Mode *Flat*

Grafik hasil output pada mode Flat menunjukkan respons frekuensi yang sangat datar dan linear di mana garis *magnitude* bertahan stabil pada angka 0 dB mulai dari frekuensi rendah hingga mendekati frekuensi 8 kHz. Hal ini mengindikasikan bahwa sinyal audio direproduksi secara murni tanpa adanya pewarnaan tambahan baik pada sektor *bass* maupun *treble*, sehingga sangat ideal untuk pemantauan referensi atau mendengarkan rekaman asli. Kurva kemudian mengalami penurunan tajam atau *rolloff* di ujung spektrum frekuensi tinggi sebagai akibat dari kerja *low pass filter* jenis IIR Butterworth yang berfungsi menghilangkan frekuensi ultra-tinggi yang tidak perlu, yang juga menyebabkan pergeseran pada respons *phase* di frekuensi tersebut.

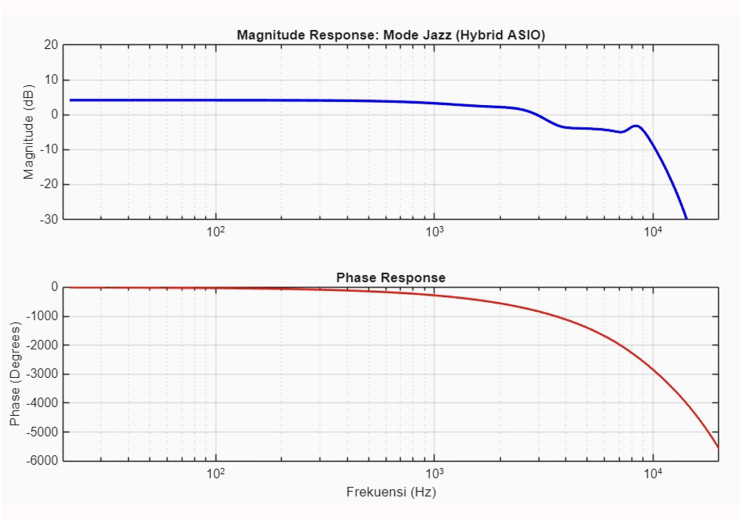
2. Mode *Rock*



Gambar 4.2 Grafik Respon Frekuensi Mode *Rock*

Untuk mode Rock, grafik output akan menampilkan karakteristik ekstrem di mana terdapat lonjakan *gain* yang tinggi pada frekuensi *bass* untuk mengakomodasi hentakan drum dan *bass guitar*, serta penurunan drastis di area *midrange* agar suara tidak terdengar tersaturasi. Sisi *treble* akan berusaha naik tinggi untuk memberikan ketajaman pada distorsi gitar dan *cymbal crash*, namun kenaikan ini akan segera berinteraksi dengan *slope* curam dari filter IIR di 8 kHz, menciptakan puncak lengkungan yang tajam di ujung spektrum frekuensi tinggi. Profil ini menghasilkan suara yang sangat energik dan dinamis, meskipun respons *phase*-nya akan mengalami pergeseran yang paling signifikan dibandingkan mode lainnya.

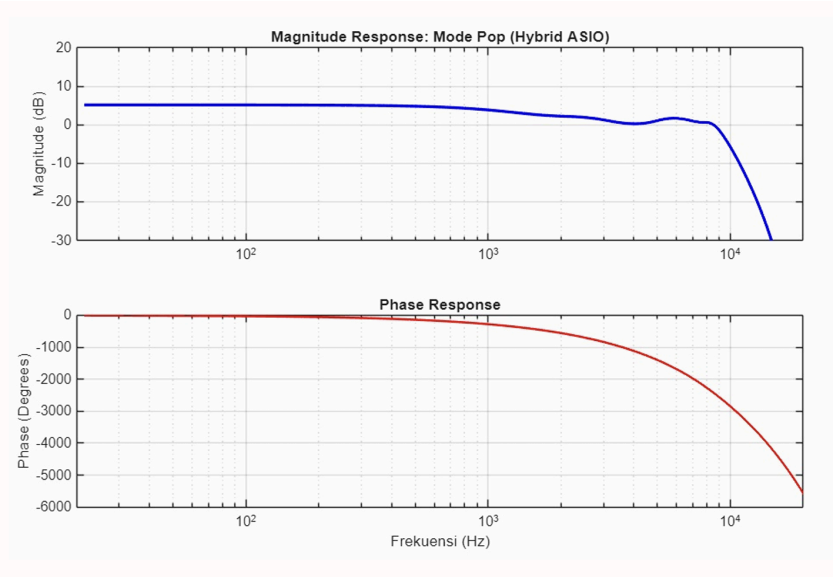
3. Mode Jazz



Gambar 4.3 Grafik Respon Frekuensi Mode *Jazz*

Hasil analisis grafik untuk mode Jazz memperlihatkan karakteristik kurva yang cenderung hangat dengan penguatan yang halus pada sektor *bass* sekitar 4 dB untuk menonjolkan suara kontra-bas dan *kick drum* yang lembut. Yang menjadi ciri khas pada grafik ini adalah penurunan *gain* atau *attenuation* yang terlihat pada rentang frekuensi menengah ke atas dan *treble*, yang sengaja dilakukan untuk meredam ketajaman instrumen tiup atau simbal agar nuansa musik terdengar lebih *mellow* dan santai. Respons *phase* pada grafik ini mengikuti perubahan amplitudo tersebut secara gradual sebelum akhirnya turun drastis saat memasuki area frekuensi *cutoff* 8 kHz.

4. Mode Pop



Gambar 4.4 Grafik Respon Frekuensi Mode *Pop*

Pada grafik mode *Pop*, terlihat adanya pola penguatan atau *boost* yang cukup merata di rentang frekuensi rendah hingga menengah bawah sekitar 5 dB yang bertujuan untuk mempertebal suara vokal dan instrumen ritmis. Kurva *magnitude* terlihat sedikit melandai di frekuensi menengah atas untuk memberikan ruang yang jernih, kemudian sedikit naik kembali untuk menambah kejernihan atau *clarity* sebelum akhirnya dipotong tajam oleh filter IIR global. Bentuk kurva seperti ini sangat umum digunakan dalam musik populer untuk membuat suara terdengar lebih bertenaga, modern, dan menonjol di telinga pendengar tanpa terdengar terlalu tajam.

BAB V
PENUTUP

A. Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, sistem Real-Time Multiband Equalizer berhasil direalisasikan menggunakan perangkat lunak MATLAB yang terintegrasi dengan Audio Interface eksternal. Penerapan arsitektur filter hibrida terbukti sangat efektif dalam pengolahan sinyal, di mana kombinasi filter FIR (N=64) yang berfungsi sebagai pembentuk karakter suara (*tone shaping*) dan filter IIR Butterworth sebagai pembersih sinyal (*noise cleaning*) mampu bekerja secara sinergis. Sistem ini juga menunjukkan performa yang stabil dengan latensi yang sangat rendah (sekitar 6,5 ms), sehingga mampu merespons perubahan parameter secara instan dan memberikan kenyamanan penggunaan yang setara dengan pemrosesan perangkat keras.

B. Saran

Demi meningkatkan fungsionalitas dan pengalaman pengguna di masa mendatang, disarankan untuk mengembangkan tampilan antarmuka berbasis grafis (GUI) menggunakan *MATLAB App Designer* agar pengoperasian sistem tidak lagi bergantung pada pengetikan kode di *Command Window*. Selain itu, fitur penyimpanan *preset* kustom perlu ditambahkan untuk memberikan fleksibilitas kepada pengguna dalam menyimpan pengaturan suara unik mereka sendiri tanpa terbatas pada *preset* bawaan. Terakhir, untuk kebutuhan integrasi yang lebih profesional, algoritma program sebaiknya dioptimalkan ke dalam bahasa C++ atau dikonversi menjadi format *plugin* VST agar dapat digunakan secara langsung dan efisien di dalam berbagai perangkat lunak *Digital Audio Workstation* (DAW).

DAFTAR PUSTAKA

[1] Pradhan, J. and INGLE, K. (2025). Development of a Digital Multi-band Audio Equalizer Using Verilog. Advanced International Journal for Research, [online] 6(5). doi:https://doi.org/10.63363/aijfr.2025.v06i05.1435.

[2] Langelaar, J., Mattsson, A. and Natvig, F. (n.d.). Development of real time audio equalizer application using MATLAB App Designer. [online] Available at: http://www.diva-portal.org/smash/get/diva2:1334188/FULLTEXT02.pdf.

[3] Antonio, M. and Ramírez, M. (2013). DESIGN AND IMPLEMENTATION OF A PARAMETRIC EQUALIZER USING IIR AND FIR FILTERS. [online] Available at: https://marquetem.wordpress.com/wp-content/uploads/2017/09/design-and-implementation-of-a-parametric-equalizer-using-iir-and-fir-filters-1.pdf.